```
import os
import matplotlib
import matplotlib.pyplot as plt

output_dir = "/content/outputs"
os.makedirs(output_dir, exist_ok=True)
_plot_counter = [0]

# ── Hook plt.show() for seaborn/matplotlib plots ──────────────────
_orig_show = plt.show
def _autosave_show(*args, **kwargs):
    for i in plt.get_fignums():
        fig = plt.figure(i)
        _plot_counter[0] += 1
        path = f"{output_dir}/plot_{_plot_counter[0]:03d}.png"
        fig.savefig(path, dpi=150, bbox_inches='tight')
        print(f"✅ Saved: plot_{_plot_counter[0]:03d}.png")
    _orig_show(*args, **kwargs)
plt.show = _autosave_show

# ── Hook sc.pl.* (Scanpy) functions ───────────────────────────────
# Scanpy saves to file via 'save=' param OR renders via plt internals.
# We patch matplotlib's Figure.show to catch ALL figure renders.
_orig_fig_show = matplotlib.figure.Figure.show
def _fig_show_patch(self, *args, **kwargs):
    _plot_counter[0] += 1
    path = f"{output_dir}/plot_{_plot_counter[0]:03d}.png"
    self.savefig(path, dpi=150, bbox_inches='tight')
    print(f"✅ Saved: plot_{_plot_counter[0]:03d}.png")
    _orig_fig_show(self, *args, **kwargs)
matplotlib.figure.Figure.show = _fig_show_patch

print("✅ Auto-save hooked for matplotlib, seaborn AND scanpy plots!")
```

```
✅ Auto-save hooked for matplotlib, seaborn AND scanpy plots!
```

```
pip install scanpy seaborn pandas openpyxl
```

```
Requirement already satisfied: statsmodels>=0.14.5 in /usr/local/lib/python3.12/dist-packages (from scanpy) (0.14.6)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from scanpy) (4.67.3)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from scanpy) (4.15.0)
Requirement already satisfied: umap-learn>=0.5.12 in /usr/local/lib/python3.12/dist-packages (from scanpy) (0.5.12)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: et-xmlfile in /usr/local/lib/python3.12/dist-packages (from openpyxl) (2.0.0)
Collecting array-api-compat>=1.7.1 (from anndata>=0.10.8->scanpy)
  Downloading array_api_compat-1.14.0-py3-none-any.whl.metadata (2.5 kB)
Collecting pandas
  Downloading pandas-2.3.3-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (91 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 91.2/91.2 kB 7.4 MB/s eta 0:00:00
```

```
Downloading session_info2-0.4.1-py3-none-any.whl (17 kB)
Downloading array_api_compat-1.14.0-py3-none-any.whl (60 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.1/60.1 kB 1.7 MB/s eta 0:00:00
Downloading scverse_misc-0.0.5-py3-none-any.whl (12 kB)
Downloading zarr-3.2.0-py3-none-any.whl (318 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 318.8/318.8 kB 9.2 MB/s eta 0:00:00
Downloading donfig-0.8.1.post1-py3-none-any.whl (21 kB)
Downloading numcodecs-0.16.5-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (9.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.2/9.2 MB 25.5 MB/s eta 0:00:00
Installing collected packages: session-info2, numcodecs, legacy-api-wrap, fast-array-utils, donfig, array-api-compat, zarr, sc
  Attempting uninstall: pandas
    Found existing installation: pandas 2.2.2
    Uninstalling pandas-2.2.2:
      Successfully uninstalled pandas-2.2.2
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is t
google-colab 1.0.0 requires pandas==2.2.2, but you have pandas 2.3.3 which is incompatible.
Successfully installed anndata-0.12.11 array-api-compat-1.14.0 donfig-0.8.1.post1 fast-array-utils-1.4.1 legacy-api-wrap-1.5 n
```

```
from __future__ import annotations

import matplotlib.pyplot as plt
import pandas as pd
import scanpy as sc
import seaborn as sns
```

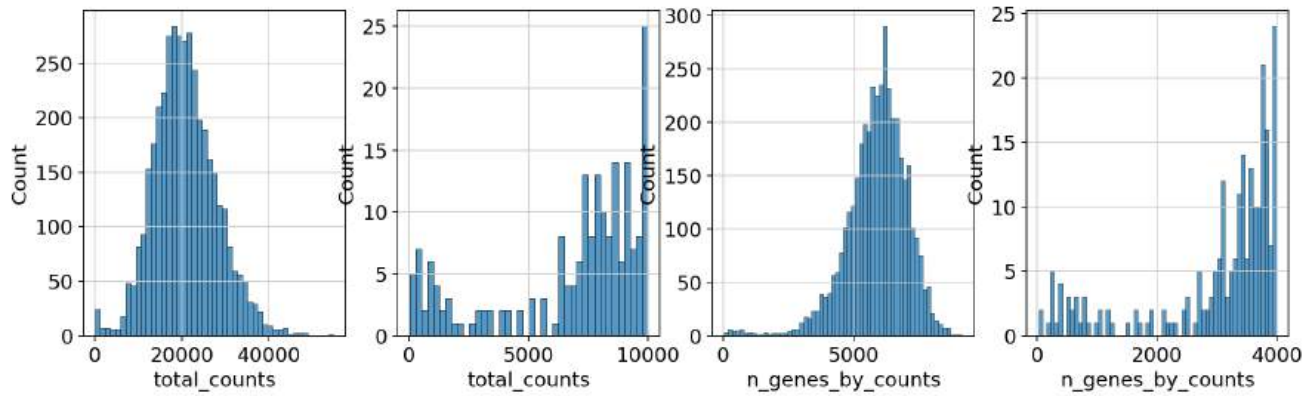
```
sc.logging.print_header()
sc.set_figure_params(facecolor="white", figsize=(8, 8))
sc.settings.verbosity = 3
```

```
adata = sc.datasets.visium_sge(sample_id="V1_Human_Lymph_Node")
adata.var_names_make_unique()
adata.var["mt"] = adata.var_names.str.startswith("MT-")
sc.pp.calculate_qc_metrics(adata, qc_vars=["mt"], inplace=True)
```

```
/tmp/ipykernel_3026/1997158664.py:1: FutureWarning: Use `squidpy.datasets.visium` instead.
  adata = sc.datasets.visium_sge(sample_id="V1_Human_Lymph_Node")
100%                                          7.86M/7.86M [00:00<00:00, 16.0MB/s]

100%                                          29.3M/29.3M [00:00<00:00, 44.3MB/s]
reading data/V1_Human_Lymph_Node/filtered_feature_bc_matrix.h5
 (0:00:01)
/usr/local/lib/python3.12/dist-packages/anndata/_core/anndata.py:1880: UserWarning: Variable names are not unique. To make them
  utils.warn_names_duplicates("var")
```

```
# Plot QC metric distributions
fig, axs = plt.subplots(1, 4, figsize=(15, 4))
sns.histplot(adata.obs["total_counts"], kde=False, ax=axs[0])
sns.histplot(
    adata.obs["total_counts"][adata.obs["total_counts"] < 10000],
    kde=False,
    bins=40,
    ax=axs[1],
)
sns.histplot(adata.obs["n_genes_by_counts"], kde=False, bins=60, ax=axs[2])
sns.histplot(
    adata.obs["n_genes_by_counts"][adata.obs["n_genes_by_counts"] < 4000],
    kde=False,
    bins=60,
    ax=axs[3],
)
```

```
<Axes: xlabel='n_genes_by_counts', ylabel='Count'>
```




```
# Filter cells based on count thresholds and mitochondrial content
sc.pp.filter_cells(adata, min_counts=5000)
sc.pp.filter_cells(adata, max_counts=35000)
adata = adata[adata.obs["pct_counts_mt"] < 20].copy()
print(f"#cells after MT filter: {adata.n_obs}")
sc.pp.filter_genes(adata, min_cells=10)
```

```
filtered out 44 cells that have less than 5000 counts
filtered out 130 cells that have more than 35000 counts
#cells after MT filter: 3861
filtered out 16916 genes that are detected in less than 10 cells
```

```
# Normalize, log-transform, and detect highly variable genes
sc.pp.normalize_total(adata, inplace=True)
sc.pp.log1p(adata)
sc.pp.highly_variable_genes(adata, flavor="seurat", n_top_genes=2000)
```

```
normalizing counts per cell
    finished (0:00:03)
extracting highly variable genes
    finished (0:00:03)
--> added
    'highly_variable', boolean vector (adata.var)
    'means', float vector (adata.var)
    'dispersions', float vector (adata.var)
    'dispersions_norm', float vector (adata.var)
```

```
!pip install igraph
```

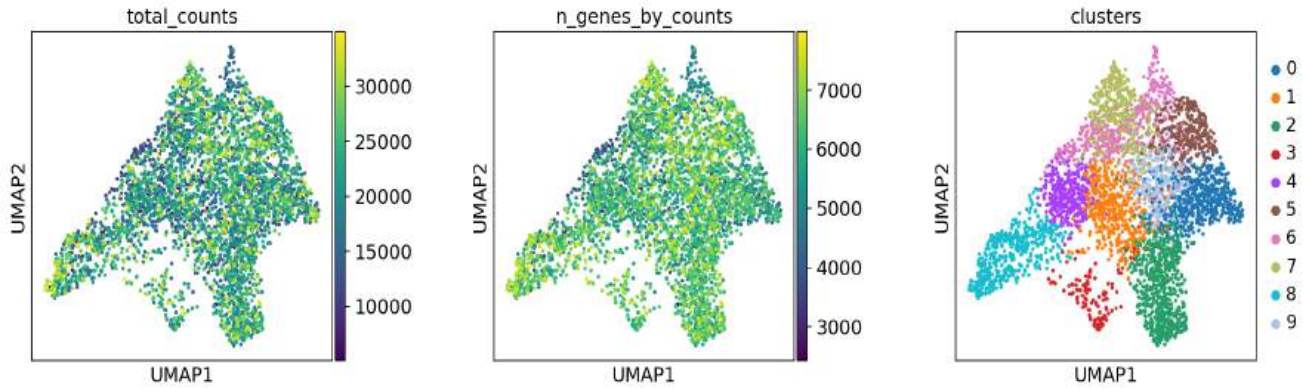
```
Collecting igraph
  Downloading igraph-1.0.0-cp39-abi3-manylinux_2_28_x86_64.whl.metadata (4.4 kB)
Collecting texttable>=1.6.2 (from igraph)
  Downloading texttable-1.7.0-py2.py3-none-any.whl.metadata (9.8 kB)
Downloading igraph-1.0.0-cp39-abi3-manylinux_2_28_x86_64.whl (5.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.7/5.7 MB 85.4 MB/s eta 0:00:00
Downloading texttable-1.7.0-py2.py3-none-any.whl (10 kB)
Installing collected packages: texttable, igraph
Successfully installed igraph-1.0.0 texttable-1.7.0
```

```
# MANIFOLD EMBEDDING AND CLUSTERING
# ==========================================================

sc.pp.pca(adata)
sc.pp.neighbors(adata)
sc.tl.umap(adata)
sc.tl.leiden(adata, key_added="clusters", flavor="igraph", directed=False, n_iterations=2)
```

```
computing PCA
    with n_comps=50
    finished (0:00:01)
computing neighbors
    using 'X_pca' with n_pcs = 50
    finished: added to `.uns['neighbors']`
    `.obsp['distances']`, distances for each pair of neighbors
    `.obsp['connectivities']`, weighted adjacency matrix (0:00:00)
computing UMAP
    finished: added
    'X_umap', UMAP coordinates (adata.obsm)
```

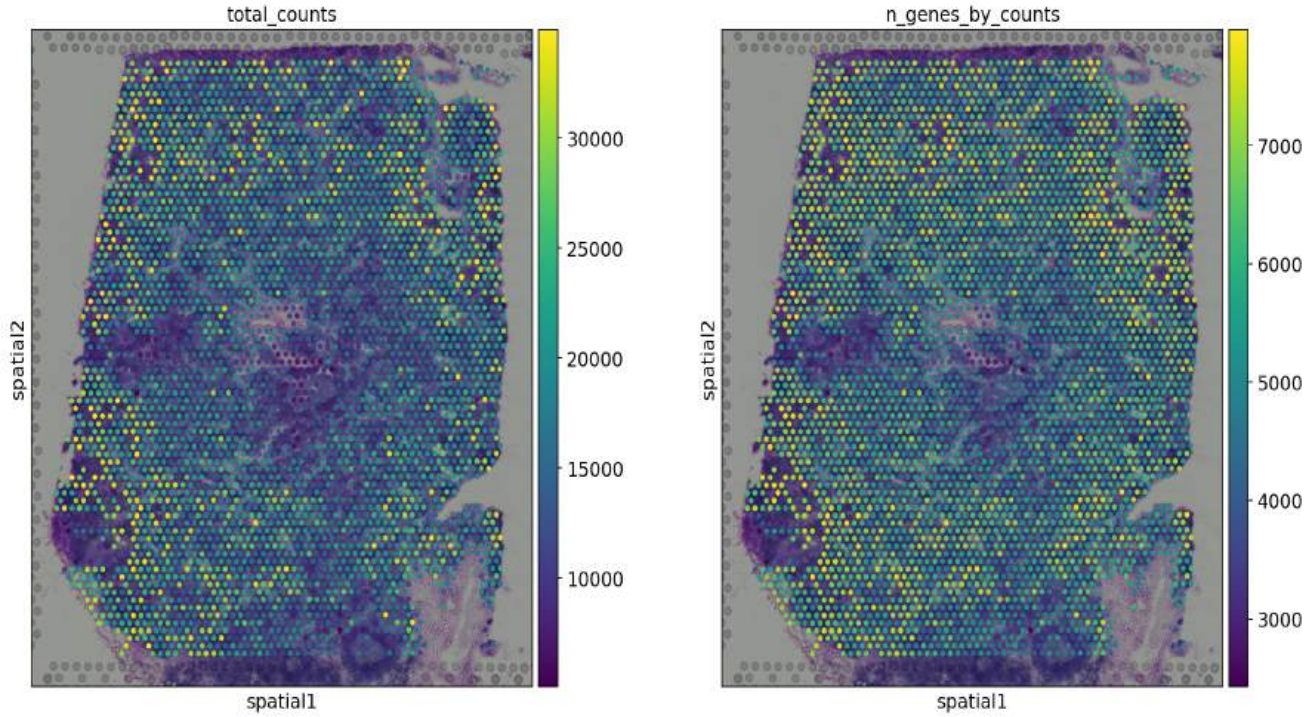
```
    'umap', UMAP parameters (adata.uns) (0:00:06)
running Leiden clustering
    finished: found 10 clusters and added
    'clusters', the cluster labels (adata.obs, categorical) (0:00:00)
```

```
# Plot UMAP colored by QC metrics and cluster labels
plt.rcParams["figure.figsize"] = (4, 4)
sc.pl.umap(adata, color=["total_counts", "n_genes_by_counts", "clusters"], wspace=0.4)
```



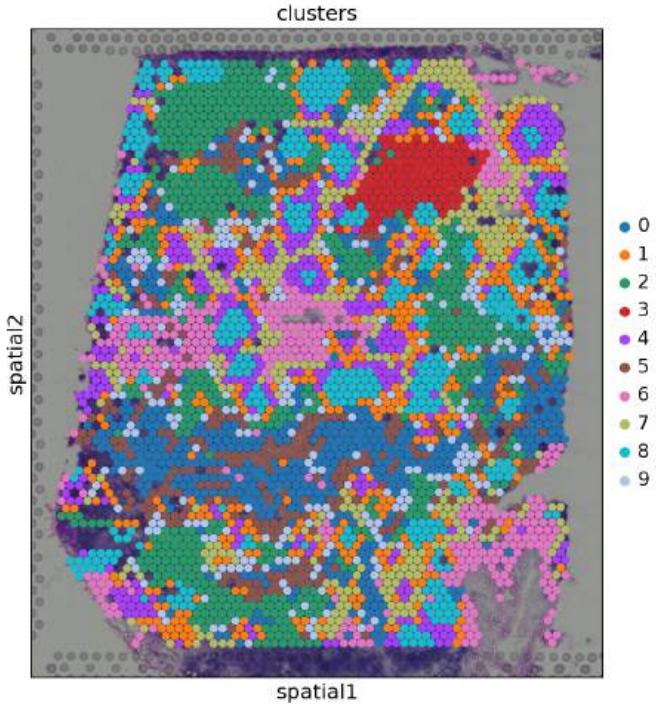

```
# VISUALIZATION IN SPATIAL COORDINATES
# ============================================================

# Overlay QC metrics on H&E tissue image
plt.rcParams["figure.figsize"] = (8, 8)
sc.pl.spatial(adata, img_key="hires", color=["total_counts", "n_genes_by_counts"])
```

```
/tmp/ipykernel_3026/1863836882.py:6: FutureWarning: Use `squidpy.pl.spatial_scatter` instead.
  sc.pl.spatial(adata, img_key="hires", color=["total_counts", "n_genes_by_counts"])
```



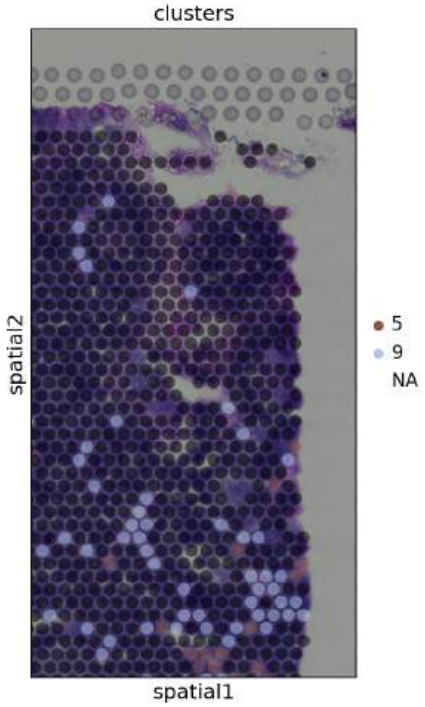

```
# Overlay cluster labels on H&E tissue image
sc.pl.spatial(adata, img_key="hires", color="clusters", size=1.5)
```

```
/tmp/ipykernel_3026/259302142.py:2: FutureWarning: Use `squidpy.pl.spatial_scatter` instead.
  sc.pl.spatial(adata, img_key="hires", color="clusters", size=1.5)
```



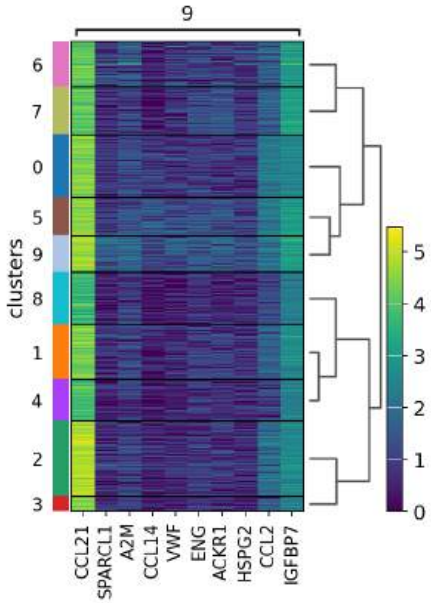

```
# Zoom into specific clusters with transparency
sc.pl.spatial(
    adata,
    img_key="hires",
    color="clusters",
    groups=["5", "9"],
    crop_coord=[7000, 10000, 0, 6000],
    alpha=0.5,
    size=1.3,
)
```

```
/tmp/ipykernel_3026/1251781545.py:2: FutureWarning: Use `squidpy.pl.spatial_scatter` instead.
  sc.pl.spatial(
```



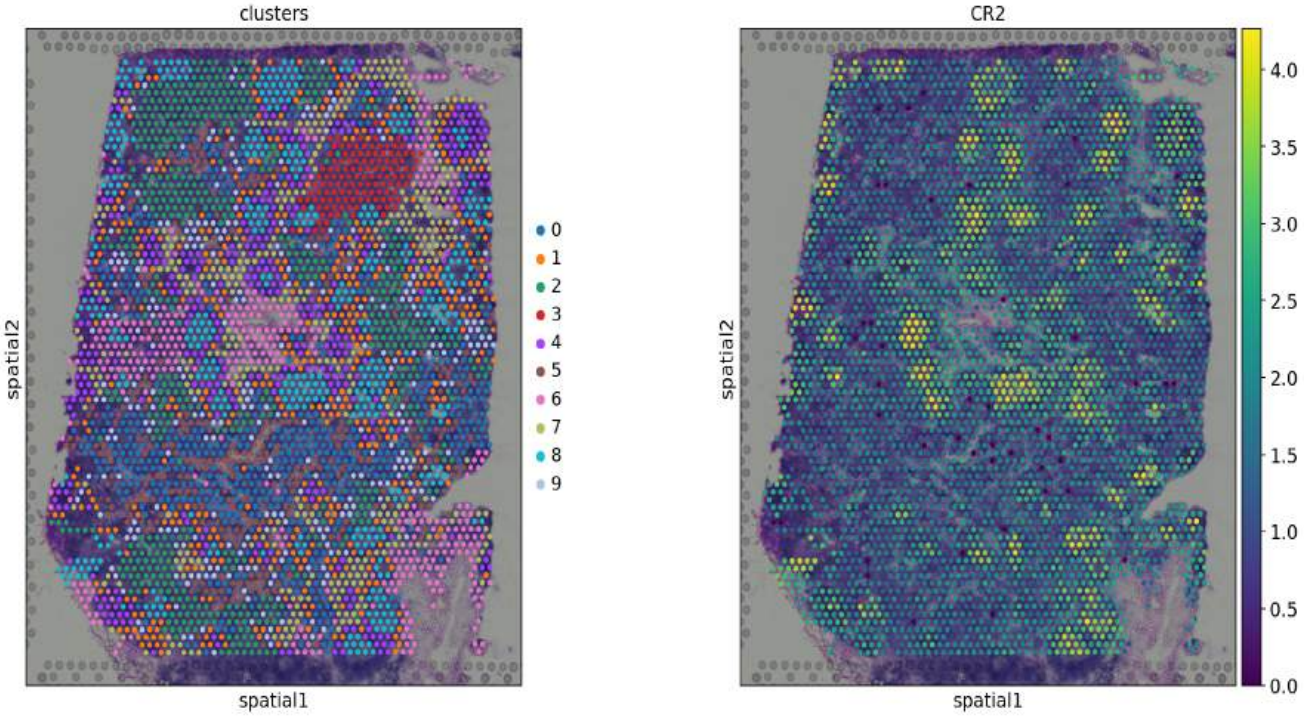
```
# CLUSTER MARKER GENES
# ============================================================

# Rank genes per cluster using t-test and plot heatmap for cluster 9
sc.tl.rank_genes_groups(adata, "clusters", method="t-test")
sc.pl.rank_genes_groups_heatmap(adata, groups="9", n_genes=10, groupby="clusters")
```

```
ranking genes
    finished: added to `.uns['rank_genes_groups']`
    'names', sorted np.recarray to be indexed by group ids
    'scores', sorted np.recarray to be indexed by group ids
    'logfoldchanges', sorted np.recarray to be indexed by group ids
    'pvals', sorted np.recarray to be indexed by group ids
    'pvals_adj', sorted np.recarray to be indexed by group ids (0:00:02)
WARNING: dendrogram data not found (using key=dendrogram_clusters). Running `sc.tl.dendrogram` with default parameters. For fine
    using 'X_pca' with n_pcs = 50
Storing dendrogram info using `.uns['dendrogram_clusters']`
WARNING: Groups are not reordered because the `groupby` categories and the `var_group_labels` are different.
categories: 0, 1, 2, etc.
var_group_labels: 9
```



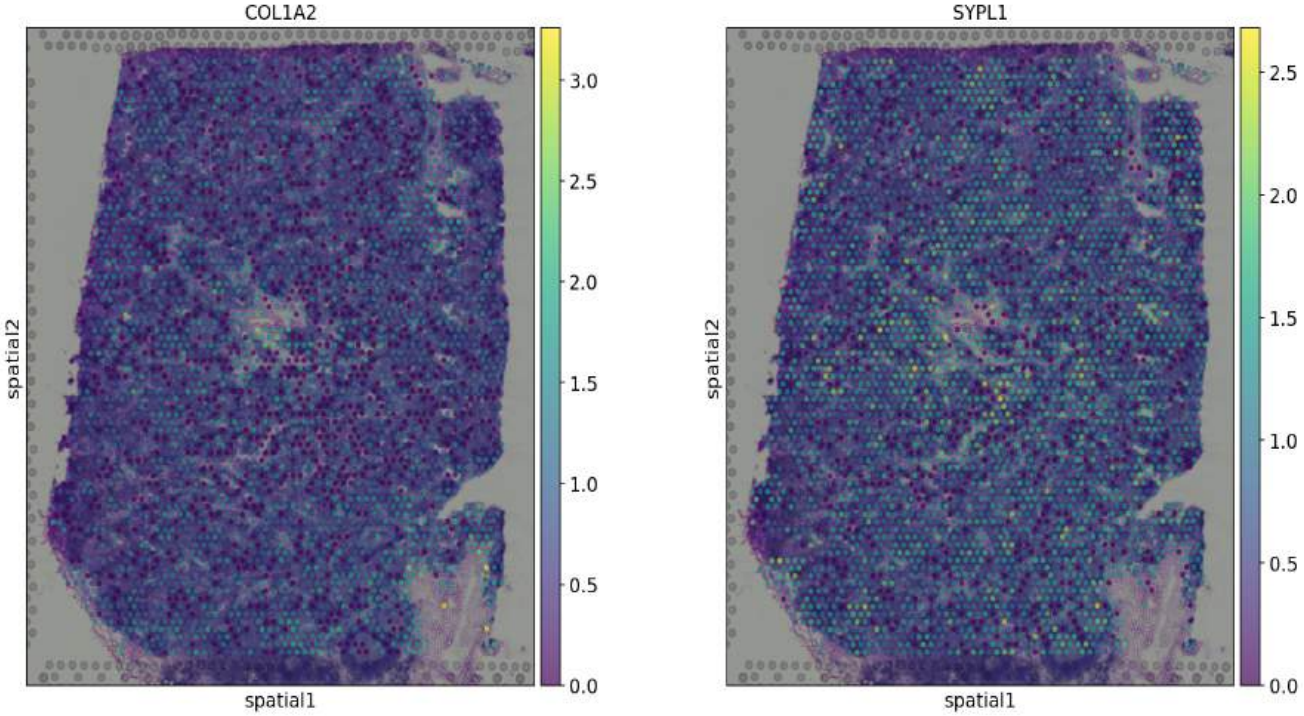

```
# Visualize top marker gene CR2 spatially
sc.pl.spatial(adata, img_key="hires", color=["clusters", "CR2"])
```

```
/tmp/ipykernel_3026/1017671507.py:2: FutureWarning: Use `squidpy.pl.spatial_scatter` instead.
  sc.pl.spatial(adata, img_key="hires", color=["clusters", "CR2"])
```




```
# Visualize additional genes of interest spatially
sc.pl.spatial(adata, img_key="hires", color=["COL1A2", "SYPL1"], alpha=0.7)
```

```
/tmp/ipykernel_3026/3294999575.py:2: FutureWarning: Use `squidpy.pl.spatial_scatter` instead.
  sc.pl.spatial(adata, img_key="hires", color=["COL1A2", "SYPL1"], alpha=0.7)
```



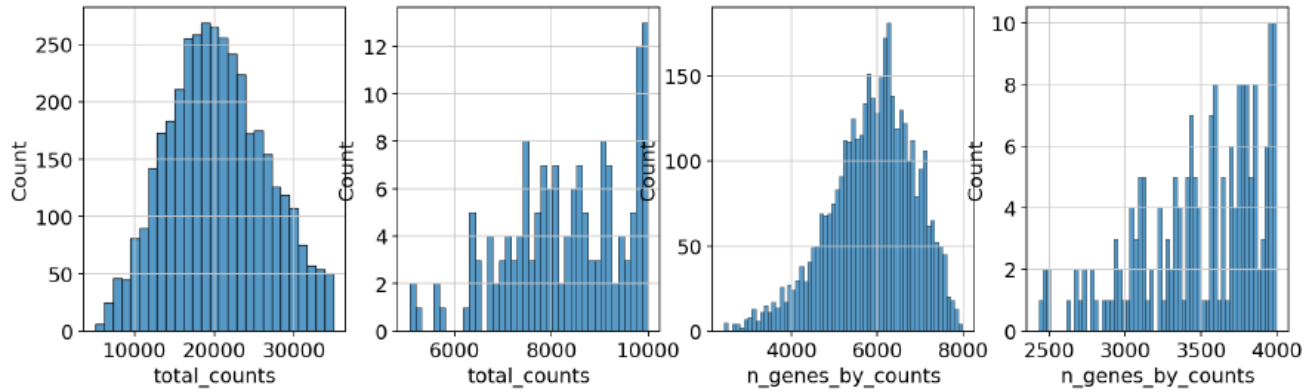

```
# MERFISH EXAMPLE
# ==========================================================

from google.colab import files
import shutil
import os
```

```
os.makedirs("./data", exist_ok=True)
uploaded = files.upload()  # select both .xlsx and .csv files

for filename in uploaded.keys():
    shutil.move(filename, f"./data/{filename}")

# Load MERFISH coordinates and counts
coordinates = pd.read_excel("./data/pnas.1912459116.sd15.xlsx", index_col=0)
counts = sc.read_csv("./data/pnas.1912459116.sd12.csv").transpose()

# Build AnnData object with spatial coordinates
adata_merfish = counts[coordinates.index, :].copy()
adata_merfish.obsm["spatial"] = coordinates.to_numpy()

# Preprocess MERFISH data
sc.pp.normalize_per_cell(adata_merfish, counts_per_cell_after=1e6)
sc.pp.log1p(adata_merfish)
sc.pp.pca(adata_merfish, n_comps=15)
sc.pp.neighbors(adata_merfish)
sc.tl.umap(adata_merfish)
sc.tl.leiden(
    adata_merfish,
    key_added="clusters",
    resolution=0.5,
    n_iterations=2,
    flavor="igraph",
    directed=False,
)

# Visualize MERFISH clusters in UMAP and spatial coordinates
sc.pl.umap(adata_merfish, color="clusters")
sc.pl.embedding(adata_merfish, basis="spatial", color="clusters")
```

Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this

```
import os, matplotlib.pyplot as plt, seaborn as sns
os.makedirs("/content/outputs", exist_ok=True)

fig, axs = plt.subplots(1, 4, figsize=(15, 4))
sns.histplot(adata.obs["total_counts"], kde=False, ax=axs[0])
sns.histplot(adata.obs["total_counts"][adata.obs["total_counts"] < 10000], kde=False, bins=40, ax=axs[1])
sns.histplot(adata.obs["n_genes_by_counts"], kde=False, bins=60, ax=axs[2])
sns.histplot(adata.obs["n_genes_by_counts"][adata.obs["n_genes_by_counts"] < 4000], kde=False, bins=60, ax=axs[3])
fig.savefig("/content/outputs/01_QC_histograms.png", dpi=150, bbox_inches='tight')
plt.show()
print("✅ Saved: 01_QC_histograms.png")
```

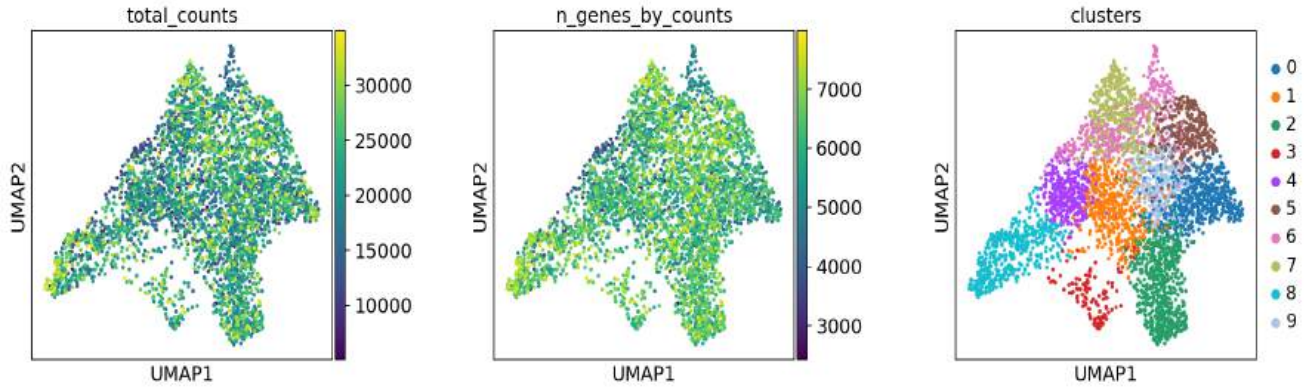
```
finished: added to `.uns['neighbors']`
    `.obsp['distances']`, distances for each pair of neighbors
    `.obsp['connectivities']`, weighted adjacency matrix (0:00:00)
✅ Saved: plot_001.png
✅ Auto-saved: plot_2.png
```




```
✅ Saved: 01_QC_histograms.png
```

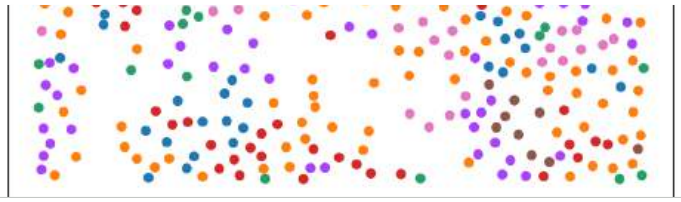
```
import os, matplotlib.pyplot as plt
os.makedirs("/content/outputs", exist_ok=True)
plt.rcParams["figure.figsize"] = (4, 4)
fig = sc.pl.umap(adata, color=["total_counts", "n_genes_by_counts", "clusters"], wspace=0.4, show=False, return_fig=True)
fig.savefig("/content/outputs/02_UMAP_clusters.png", dpi=150, bbox_inches='tight')
plt.show()
print("✅ Saved: 02_UMAP_clusters.png")
```

```
✅ Saved: plot_003.png
✅ Auto-saved: plot_4.png
```

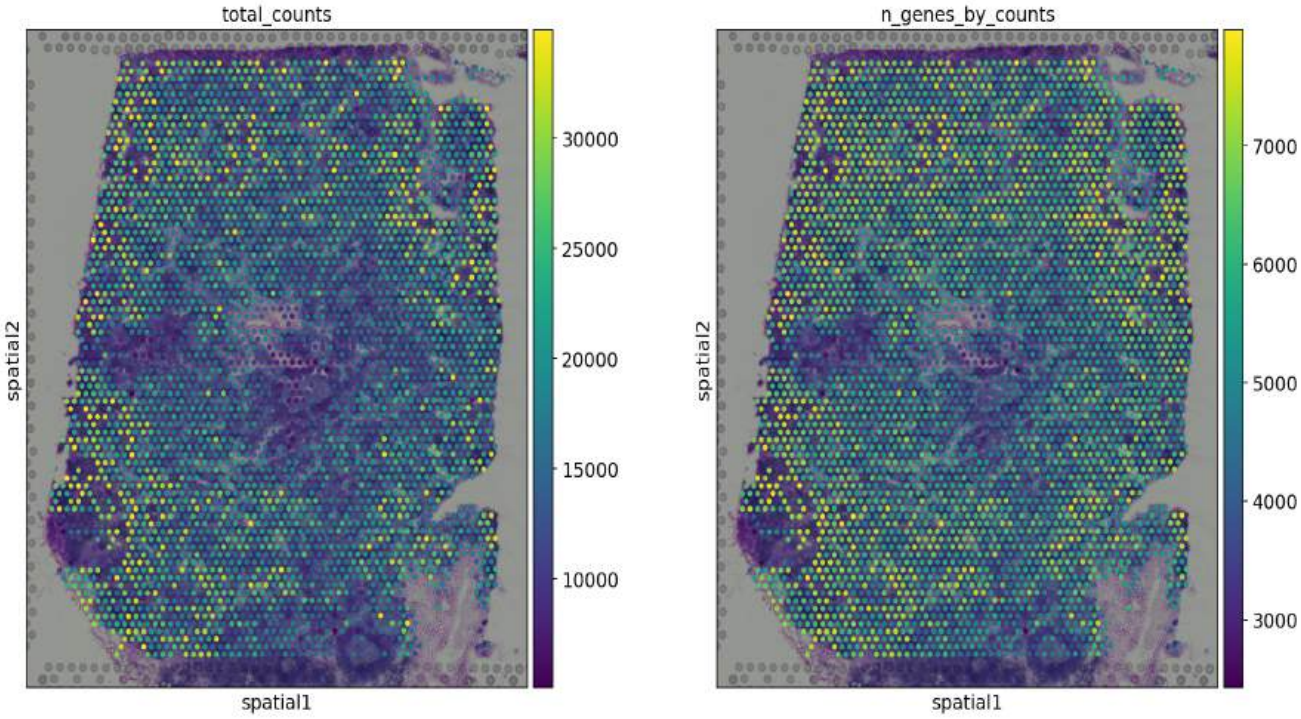



```
✅ Saved: 02_UMAP_clusters.png
```

```
import os, matplotlib.pyplot as plt
os.makedirs("/content/outputs", exist_ok=True)
plt.rcParams["figure.figsize"] = (8, 8)
fig = sc.pl.spatial(adata, img_key="hires", color=["total_counts", "n_genes_by_counts"], show=False, return_fig=True)
fig.savefig("/content/outputs/03_spatial_QC.png", dpi=150, bbox_inches='tight')
plt.show()
print("✅ Saved: 03_spatial_QC.png")
```



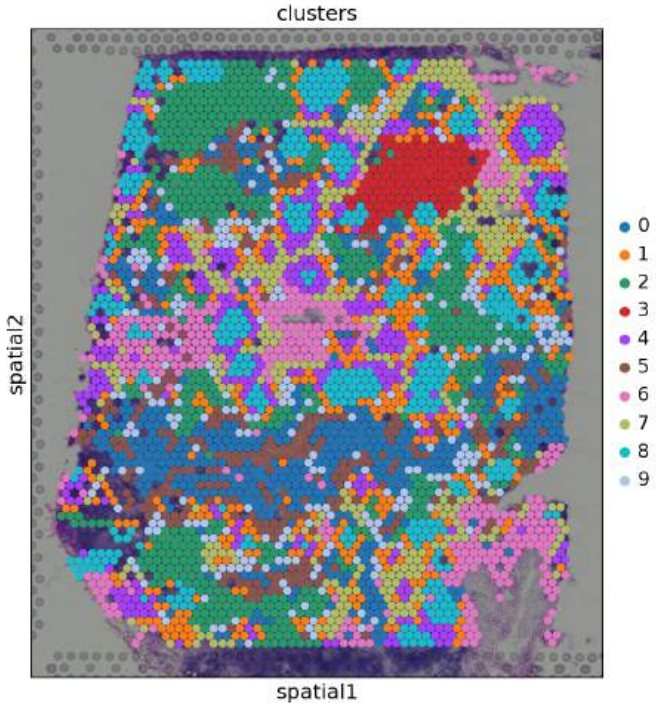
```
/tmp/ipykernel_3026/3944176952.py:4: FutureWarning: Use `squidpy.pl.spatial_scatter` instead.
  fig = sc.pl.spatial(adata, img_key="hires", color=["total_counts", "n_genes_by_counts"], show=False, return_fig=True)
✅ Saved: plot_005.png
✅ Auto-saved: plot_6.png
```




```
✅ Saved: 03_spatial_QC.png
```

```
import os, matplotlib.pyplot as plt
os.makedirs("/content/outputs", exist_ok=True)
fig = sc.pl.spatial(adata, img_key="hires", color="clusters", size=1.5, show=False, return_fig=True)
fig.savefig("/content/outputs/04_spatial_clusters.png", dpi=150, bbox_inches='tight')
plt.show()
print("✅ Saved: 04_spatial_clusters.png")
```

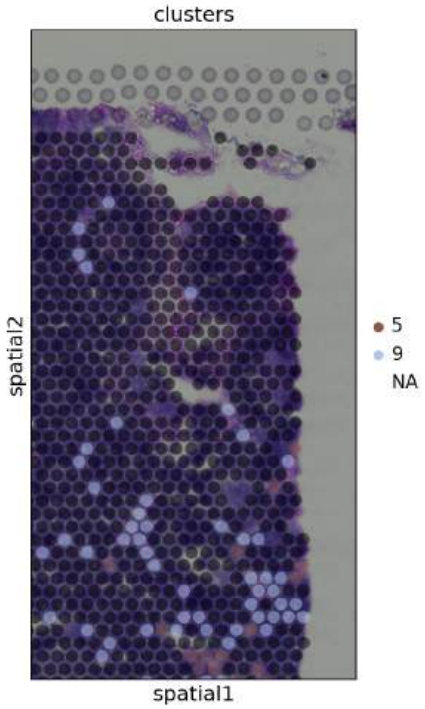
```
/tmp/ipykernel_3026/1824777175.py:3: FutureWarning: Use `squidpy.pl.spatial_scatter` instead.
  fig = sc.pl.spatial(adata, img_key="hires", color="clusters", size=1.5, show=False, return_fig=True)
✅ Saved: plot_007.png
✅ Auto-saved: plot_8.png
```




```
✅ Saved: 04_spatial_clusters.png
```

```
import os, matplotlib.pyplot as plt
os.makedirs("/content/outputs", exist_ok=True)
fig = sc.pl.spatial(adata, img_key="hires", color="clusters", groups=["5", "9"],
    crop_coord=[7000, 10000, 0, 6000], alpha=0.5, size=1.3, show=False, return_fig=True)
fig.savefig("/content/outputs/05_spatial_zoom_5_9.png", dpi=150, bbox_inches='tight')
plt.show()
print("✅ Saved: 05_spatial_zoom_5_9.png")
```

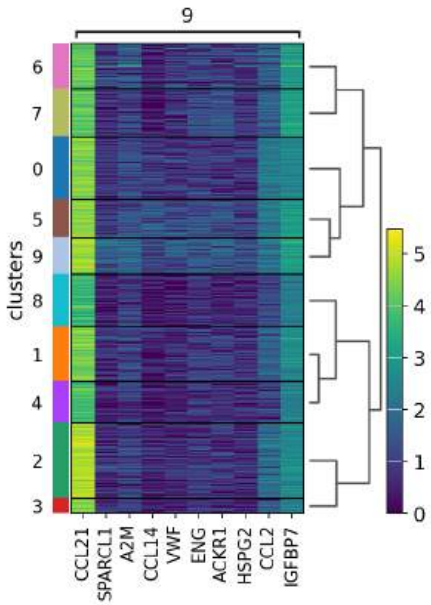
```
/tmp/ipykernel_3026/4291890227.py:3: FutureWarning: Use `squidpy.pl.spatial_scatter` instead.
  fig = sc.pl.spatial(adata, img_key="hires", color="clusters", groups=["5", "9"],
✅ Saved: plot_009.png
✅ Auto-saved: plot_10.png
```




```
✅ Saved: 05_spatial_zoom_5_9.png
```

```
import os, matplotlib.pyplot as plt
os.makedirs("/content/outputs", exist_ok=True)

sc.pl.rank_genes_groups_heatmap(adata, groups="9", n_genes=10, groupby="clusters", show=False)
plt.savefig("/content/outputs/06_heatmap_cluster9.png", dpi=150, bbox_inches='tight')
plt.show()
print("✅ Saved: 06_heatmap_cluster9.png")
```

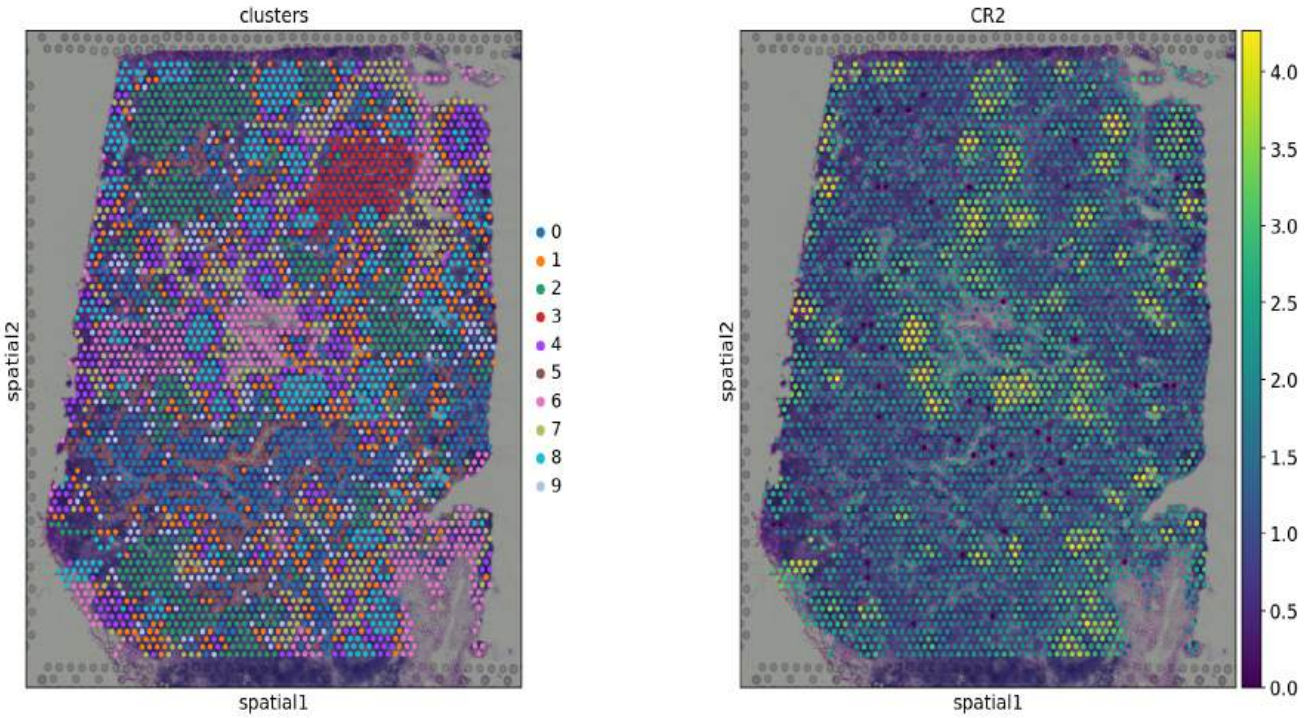
```
WARNING: Groups are not reordered because the `groupby` categories and the `var_group_labels` are different.
categories: 0, 1, 2, etc.
var_group_labels: 9
✅ Saved: plot_011.png
✅ Auto-saved: plot_12.png
```




```
✅ Saved: 06_heatmap_cluster9.png
```

```
import os, matplotlib.pyplot as plt
os.makedirs("/content/outputs", exist_ok=True)
fig = sc.pl.spatial(adata, img_key="hires", color=["clusters", "CR2"], show=False, return_fig=True)
fig.savefig("/content/outputs/07_spatial_CR2.png", dpi=150, bbox_inches='tight')
plt.show()
print("✅ Saved: 07_spatial_CR2.png")
```

```
/tmp/ipykernel_3026/215317072.py:3: FutureWarning: Use `squidpy.pl.spatial_scatter` instead.
  fig = sc.pl.spatial(adata, img_key="hires", color=["clusters", "CR2"], show=False, return_fig=True)
✅ Saved: plot_013.png
✅ Auto-saved: plot_14.png
```

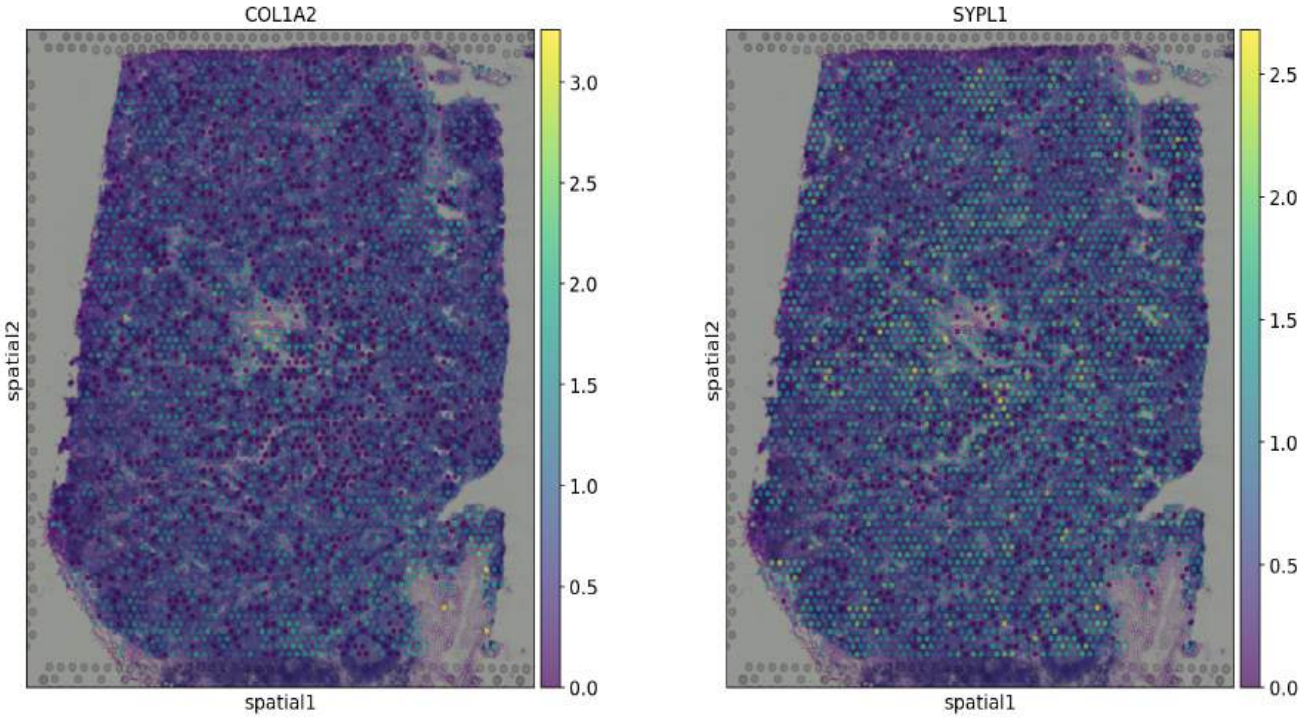



```
✅ Saved: 07_spatial_CR2.png
```

```
import os, matplotlib.pyplot as plt
os.makedirs("/content/outputs", exist_ok=True)
```

```
fig = sc.pl.spatial(adata, img_key="hires", color=["COL1A2", "SYPL1"], alpha=0.7, show=False, return_fig=True)
fig.savefig("/content/outputs/08_spatial_COL1A2_SYPL1.png", dpi=150, bbox_inches='tight')
plt.show()
```

```
/tmp/ipykernel_3026/3235333924.py:3: FutureWarning: Use `squidpy.pl.spatial_scatter` instead.
  fig = sc.pl.spatial(adata, img_key="hires", color=["COL1A2", "SYPL1"], alpha=0.7, show=False, return_fig=True)
✅ Saved: plot_015.png
✅ Auto-saved: plot_16.png
```




```
✅ Saved: 08_spatial_COL1A2_SYPL1.png
```

```
import os, matplotlib.pyplot as plt
os.makedirs("/content/outputs", exist_ok=True)

fig = sc.pl.umap(adata_merfish, color="clusters", show=False, return_fig=True)
fig.savefig("/content/outputs/09_MERFISH_UMAP.png", dpi=150, bbox_inches='tight')
plt.show()
print("✅ Saved: 09_MERFISH_UMAP.png")

fig = sc.pl.embedding(adata_merfish, basis="spatial", color="clusters", show=False, return_fig=True)
fig.savefig("/content/outputs/10_MERFISH_spatial.png", dpi=150, bbox_inches='tight')
```